

Python Async/Await and Coroutines Guide

Table of Contents

1. [Core Concepts Overview](#)
 2. [Regular Generators](#)
 3. [Regular Coroutines](#)
 4. [Async Generators](#)
 5. [Custom Awaitables](#)
 6. [Iterables, Iterators, and Generators](#)
 7. [Generator-based Coroutines](#)
 8. [Concurrent Execution](#)
 9. [Unsync Library](#)
 10. [Generator Priming](#)
-

Core Concepts Overview

Concept	await?	next()?	Notes
<code>def + yield</code>	✗	✓	Regular generators
<code>async def + await</code>	✓	✗	Regular coroutines
<code>async def + yield</code>	✗	✓ via <code>async for</code>	Async generators
<code>__await__()</code> object	✓	✓ if it's a generator	Custom awaitables

Regular Generators

Definition: `def + yield`

A regular generator (synchronous). You can `next()` it, but you can't `await` it.

python

```
def gen():
    yield "hello"
    yield "world"

g = gen()
print(next(g)) # ✅ "hello"
print(next(g)) # ✅ "world"

# await g ❌ TypeError: cannot 'await' a generator object
```

Regular Coroutines

Definition: `async def + await`

A coroutine function. You must use `await`, but you can't `next()` it.

```
python

import asyncio

async def say_hello():
    return "Hello Async"

async def main():
    result = await say_hello() # ✅ works
    print(result)

asyncio.run(main())

# say_hello() is a coroutine
# next(say_hello()) ❌ TypeError
```

Important: You cannot use `await` in a function that is not declared with `async def`. This will result in a `SyntaxError`.

```
python

def regular_function():
    await some_coroutine() # ❌ SyntaxError: 'await' outside async function
```

Async Generators

Definition: `async def + yield`

This defines an async generator. You can't `await` it, but you can iterate with `async for`.

```
python
```

```
import asyncio
```

```
async def async_gen():
```

```
    for i in range(3):
```

```
        yield i # ✅ allowed in async generators
```

```
async def main():
```

```
    async for val in async_gen(): # ✅ works
```

```
        print(val)
```

```
asyncio.run(main())
```

```
# await async_gen() ❌ TypeError: can't be awaited
```

```
# next(async_gen()) ❌ TypeError
```

Custom Awaitables

Definition: Object with `__await__()`

Custom object that is awaitable. This can be `await`-ed and sometimes also `next()`-ed.

```
python
```

```

class CustomAwaitable:
    def __await__(self):
        print("Inside __await__")
        yield # pause here
        return "Done!"

async def main():
    result = await CustomAwaitable() # ✅ Yes, works
    print(result)

asyncio.run(main())

# Technically, you could also do:
g = CustomAwaitable().__await__()
print(next(g)) # ✅ works because __await__ returns a generator

```

Why Does This Work?

Python allows any object with a `__await__()` method to be `await`-ed. According to PEP 492 (async/await), an object is awaitable if:

- It is a coroutine object, OR
- It has a `__await__()` method that returns an iterator

What Python Does Internally

When you write `await CustomAwaitable()`:

1. Calls `CustomAwaitable().__await__()` → this returns a generator
 2. That generator is driven by the event loop, as if by repeated `next()` calls
 3. The `yield` inside the generator causes it to pause
 4. When the generator is complete, the return value ("Done!") is returned from `await`
-

Iterables, Iterators, and Generators

Quick Reference

Term	Is it Iterable?	Is it Iterator?	Can use for loop?	Can use next()?
Iterable	✔ Yes	✗ Not necessarily	✔ Yes	✗ No (unless it's also an iterator)
Iterator	✔ Yes	✔ Yes	✔ Yes	✔ Yes
Generator	✔ Yes	✔ Yes	✔ Yes	✔ Yes

Definitions

1. Iterable

- An object that can be looped over (e.g., `for x in obj`)
- Must implement `__iter__()`, which returns an iterator
- Examples: list, tuple, dict, str, generator, custom classes with `__iter__`

2. Iterator

- An object that produces values one at a time using `__next__()`
- Must implement both:
 - `__iter__()` → returns self
 - `__next__()` → returns the next item (raises `StopIteration` at the end)

3. Generator

- A special kind of iterator created using a function with `yield`
- Automatically implements `__iter__()` and `__next__()`
- Maintains internal state between calls

Key Differences

Feature	Iterable	Iterator
Used in for	✔ Yes	✔ Yes
Has <code>__iter__()</code>	✔ Yes	✔ Yes
Has <code>__next__()</code>	✗ Not required	✔ Yes
Can be reused	✔ Usually (e.g. list)	✗ No — exhausted after use
Needs <code>iter()</code>	✔ Yes	✗ Already ready

Examples

Iterable but Not Iterator

python

```
lst = [1, 2, 3]      # list is iterable  
it = iter(lst)       # this is now an iterator
```

```
print(hasattr(lst, '__next__')) # False  
print(hasattr(it, '__next__')) # True
```

Generator is Iterator & Iterable

python

```
def gen():  
    yield 1
```

```
g = gen()
```

```
print(hasattr(g, '__iter__')) # True  
print(hasattr(g, '__next__')) # True
```

Generator-based Coroutines

Using `@coroutine` Decorator

python

```
from collections import deque
```

```
from types import coroutine
```

```
friends = deque(['Ram', 'Sita', 'Laxman', 'Shatrughan', 'Bharat', 'Hanuman'])
```

```
@coroutine
```

```
def friend_upper():
```

```
    while friends:
```

```
        friend = friends.popleft().upper()
```

```
        greeting = yield
```

```
        print(f'{greeting} {friend}')
```

```
async def greet(g):
```

```
    await g
```

```
greeter = greet(friend_upper())
```

```
greeter.send(None)
```

```
greeter.send('Hello')
```

```
greeter.send('Hi')
```

How `@coroutine` Works

The `@coroutine` decorator from `types` transforms a generator-based coroutine into something that can be used with `await`:

- It adds a `__await__()` method to the generator object
- This makes it legal to use with `await`
- The generator object now behaves like an awaitable object

This is how asyncio used to work before `async def`/`await` syntax was introduced (Python 3.4 and earlier Python 3.5 styles).

Concurrent Execution

Sequential vs Concurrent

Sequential Execution (No Concurrency)

```
python
```

```
async def main():
    await task1() # waits until task1 is fully done
    await task2() # starts only after task1 is done
```

Concurrent Execution Methods

Method 1: Using `asyncio.gather()`

```
python

async def main():
    await asyncio.gather(task1(), task2()) # ✔ runs concurrently
```

Method 2: Using `asyncio.create_task()`

```
python

async def main():
    t1 = asyncio.create_task(task1())
    t2 = asyncio.create_task(task2())
    await t1
    await t2
```

Summary

Pattern	Concurrent?	Notes
<code>await task1(); await task2()</code>	✗ No	Runs one after the other
<code>await asyncio.gather(task1(), task2())</code>	✔ Yes	Best for launching coroutines together
<code>create_task + await</code>	✔ Yes	Lower-level, used for long-lived tasks

Key Principle: `await` = wait for one thing, `asyncio.gather`/`create_task` = start multiple things, then await them together

Unsync Library

The `unsync` library provides a unified interface for running synchronous, asynchronous, and CPU-bound functions concurrently.

Overview

Type of Task	Decorator Syntax	Backed by	Returns
Sync I/O-bound (default)	@unsync	ThreadPoolExecutor	UnsyncFuture
CPU-bound (heavy compute)	@unsync(process=True)	ProcessPoolExecutor	UnsyncFuture
Async coroutine	@unsync on async def	asyncio event loop	UnsyncFuture

Complete Example

python

```

from unsync import unsync
import time
import asyncio
import math

# --- Threaded: I/O-style sleep ---
@unsync
def sleep_task(name):
    time.sleep(1)
    return f"Done sleeping: {name}"

# --- Async: asyncio-style task ---
@unsync
async def async_task(name):
    await asyncio.sleep(1)
    return f"Done async: {name}"

# --- Process: CPU-bound task ---
@unsync(process=True)
def cpu_task(x):
    # Simulate CPU-intensive operation
    return f"CPU result: sqrt({x}) = {math.sqrt(x)}"

# --- Run all tasks together ---
def main():
    futures = [
        sleep_task("Threaded"),    # Threaded function
        async_task("Async"),       # Async coroutine
        cpu_task(1000000),         # CPU-bound process
    ]

    # Wait for all to complete and collect results
    results = [f.result() for f in futures]
    print("Results:")
    for r in results:
        print(r)

if __name__ == '__main__':
    main()

```

UnsyncFuture Methods

Method/Property	Meaning
<code>.result()</code>	Wait and return the result (blocking)
<code>.done()</code>	True if completed
<code>.exception()</code>	Returns exception if raised
<code>.add_done_callback(fn)</code>	Call when future completes

Generator Priming

Can You Prime a Coroutine with `next()`?

Answer: Yes, but only for generator-based coroutines, not `async def` coroutines.

Generator-based Coroutine (`@coroutine + yield`)

✅ **Yes**, you can prime it using `next()`:

```
python

from types import coroutine

@coroutine
def gen_coro():
    print("Primed")
    yield "Paused here"

g = gen_coro()
print(next(g)) # ✅ works: "Primed" then yields "Paused here"
```

Native Coroutine (`async def`)

❌ **No**, you cannot use `next()`:

```
python

async def native_coro():
    print("Inside")
    return 42

c = native_coro()
next(c) # ❌ TypeError: 'coroutine' object is not an iterator
```

Summary Table

Type of Function	Can you <code>next()</code> it?	Can you <code>await</code> it?
<code>def + yield</code>	✅ Yes	❌ No
<code>@coroutine + yield</code>	✅ Yes	✅ Yes
<code>async def</code>	❌ No	✅ Yes
<code>async def + yield</code>	❌ No (not iterable)	❌ No (<code>async for</code> only)

`next()` vs `send()` Interchangeability

For most generators, you can use `next()` and `send(None)` interchangeably, but there are important nuances:

When They're Equivalent

python

```
def my_gen():
    yield "step 1"
    yield "step 2"

g = my_gen()
print(next(g))      # ✅ "step 1"
print(g.send(None)) # ✅ "step 2" — works like next()
```

When They're Not Interchangeable

Case 1: First call to a generator with `yield =` expression

python

```
def my_gen():
    name = yield "What's your name?"
    yield f"Hello, {name}"

g = my_gen()
print(next(g))      # ✅ primes the generator
print(g.send("Alice")) # ✅ sends "Alice" into the generator

# But this will fail:
g = my_gen()
g.send("Alice") # ❌ TypeError: can't send non-None value to a just-started generator
```

Case 2: When you want to inject a value

python

```
def echo():  
    while True:  
        val = yield  
        print(f"Received: {val}")  
  
g = echo()  
next(g)      # ✅ primes to first `yield`  
g.send("Hello") # ✅ sends "Hello" into the generator
```

Summary Table

Use Case	Use <code>next()</code>	Use <code>send(None)</code>	Use <code>send(value)</code>
Prime a new generator	✅ Yes	✅ Yes	❌ No
Resume at a bare yield	✅ Yes	✅ Yes	⚠️ Only if value is None
Pass value to yield expression	❌ No	❌ No	✅ Yes

Key Points:

- `next()` is shorthand for `send(None)`
- You must use `next()` or `send(None)` to prime a generator (first step)
- Use `send(value)` only after priming, when the generator is paused at `name = yield`
- Never use `send(value)` on a just-started generator — you'll get a `TypeError`

Conclusion

Python's async/await system provides powerful tools for concurrent programming:

- **Generators** (`def + yield`) for lazy iteration and data pipelines
- **Coroutines** (`async def + await`) for asynchronous I/O operations
- **Async generators** (`async def + yield`) for asynchronous iteration
- **Custom awaitables** (objects with `__await__()`) for advanced use cases

Understanding these concepts and their interactions is crucial for writing efficient, concurrent Python code. The `unsync` library provides a convenient way to unify different concurrency models, while proper use of `asyncio.gather()` and `create_task()` enables true concurrent execution.

Remember: `await` means "wait for one thing," while tools like `gather()` and `create_task()` allow you to start multiple things and then await them together for true concurrency.